

Параллелизмы в верификации нейронных сетей

Дипломная работа

Воробьев Сергей Юрьевич

Научный руководитель:

Ильюшин Евгений Альбинович

Московский государственный университет имени М. В. Ломоносова

Факультет вычислительной математики и кибернетики

Кафедра информационной безопасности

23 января 2026 г.



Содержание

Задача верификации нейронных сетей

Распараллеливание верификации

Выбор метода параллелизма

Реализация

Планируемые эксперименты



Формальная верификация нейронных сетей

Задача

Доказать, что для любого входа $x \in C$ (область возмущений), выход сети $f(x)$ принадлежит безопасному множеству S .

Пример: Верификация устойчивости к состязательным атакам

- ▶ Дано: изображение x , модель f , радиус возмущения ϵ
- ▶ Проверить: для всех $x' : \|x - x'\|_\infty \leq \epsilon$ выполняется $\arg \max f(x) = \arg \max f(x')$



Методы верификации: CROWN

Алгоритм CROWN

Находит линейные границы выхода сети через обратное распространение:

$$\underline{W}_j x + \underline{b}_j \leq f_j(x) \leq \overline{W}_j x + \overline{b}_j$$

Ключевая операция: $A_{prev} = A_{curr} \cdot W$

- ▶ A — матрица символьных границ, размер:
($Batch \times Spec \times Neurons$)
- ▶ Для глубоких сетей матрицы A занимают огромный объем памяти
- ▶ **AlphaBetaCROWN** — state-of-the-art верификатор
(победитель VNN-COMP 2021-2023)



Проблема масштабируемости

Барьер памяти (Memory Wall)

Размер матриц A растет как $O(Batch \times Spec \times Neurons)$

Пример: Для трансформера с $N = 4096$, $B = 128$, $S = 10$:

- ▶ Одна матрица A : ~ 20 МБ
- ▶ Для всех слоев: может превысить 80 ГБ на A100

Разрыв возможностей

- ▶ **Обучение:** модели на миллиарды параметров, тысячи GPU
- ▶ **Верификация:** ограничена памятью одного GPU



Текущее состояние: Data Parallelism

AlphaBetaCROWN использует параллелизм по данным

Алгоритм Branch-and-Bound разбивает входное пространство на подзадачи, каждая решается на отдельном GPU

Ограничение:

- ▶ Решает проблему **времени вычислений**
- ▶ НЕ решает проблему **памяти**
- ▶ Если одна задача не влезает в память одного GPU — верификация невозможна

Необходимость модельного параллелизма

Нужно разделить саму модель и матрицы A между GPU



Опыт из обучения моделей

Три основных подхода к распределенному обучению:

1. Tensor Parallelism (TP)

- ▶ Разделение матриц внутри слоя
- ▶ Megatron-LM, широко используется для LLM

2. Pipeline Parallelism (PP)

- ▶ Разделение слоев модели по глубине
- ▶ GPipe, используется для очень глубоких сетей

3. Fully Sharded Data Parallelism (FSDP)

- ▶ Шардирование параметров и оптимизатора
- ▶ FSDP (PyTorch), ZeRO (DeepSpeed)

Вопрос: Какой подход применим к верификации?



Математика обратного распространения границ

Ключевая операция CROWN

$$A_{prev} = A_{curr} \cdot W$$

где A — матрица символьных границ, W — веса слоя

Для линейного слоя:

- ▶ A_{curr} : размер $(B \times S \times N_{out})$
- ▶ W : размер $(N_{out} \times N_{in})$
- ▶ A_{prev} : размер $(B \times S \times N_{in})$

Аналогия с обучением

Это похоже на обратное распространение, но вместо градиентов мы передаем **символьные матрицы границ**



Сравнение подходов

Характеристика	TP	PP	FSDP
Память на GPU	$O(M/P)$	$O(M/P)$	$O(M)$
Коммуникации	Высокие	Высокие	Низкие
Шардирует A -матрицы	Да	Нет	Нет
Подходит для VaB	Да	Нет	Нет
Сложность реализации	Высокая	Очень высокая	Средняя

Вывод: Tensor Parallelism — оптимальный выбор для верификации



Tensor Parallelism: Column Parallel

Column Parallel слой

Веса разделены по **выходной** размерности

Прямой проход: $Y = X \cdot W$

- ▶ $W = [W_0 | W_1]$ разделен между GPU
- ▶ $Y = [Y_0 | Y_1]$ также разделен
- ▶ Требуется AllGather для получения полного Y

Обратный CROWN: $A_{prev} = A_{curr} \cdot W$

- ▶ A_{curr} уже разделена: $[A_0 | A_1]$
- ▶ Локально: $P_i = A_i \cdot W_i$ (частичное произведение)
- ▶ **AllReduce:** $A_{prev} = P_0 + P_1$ на всех GPU



Tensor Parallelism: Row Parallel

Row Parallel слой

Веса разделены по **входной** размерности

Прямой проход: $Y = X \cdot W$

- ▶ $X = [X_0|X_1]$ разделен между GPU
- ▶ Локально: $P_i = X_i \cdot W_i$
- ▶ AllReduce: $Y = P_0 + P_1$

Обратный CROWN: $A_{prev} = A_{curr} \cdot W$

- ▶ A_{curr} реплицирована (полная матрица на всех GPU)
- ▶ Локально: $A_{prev,i} = A_{curr} \cdot W_i$
- ▶ Результат автоматически разделен
- ▶ **Коммуникация не нужна!**



Почему не Pipeline Parallelism?

Проблемы РР для верификации

1. **Skip connections** в ResNet/Transformer требуют передачи матриц A между GPU
2. **Двунаправленные зависимости:**
 - ▶ Прямой проход: для вычисления границ ReLU
 - ▶ Обратный CROWN: для распространения A
3. **Bubbles:** РР эффективен только при большом количестве микро-батчей, но в верификации одного изображения такого нет
4. НЕ шардирует матрицы A — проблема памяти остается!

Вывод: РР применим только для очень глубоких сетей без skip connections



Почему не FSDP?

Проблемы FSDP для верификации

1. В режиме inference (верификация) нет:
 - ▶ Градиентов
 - ▶ Состояний оптимизатора
2. Веса модели обычно малы по сравнению с матрицами A
3. FSDP НЕ шардирует активации и промежуточные матрицы A
4. Решает проблему хранения весов, но игнорирует главную проблему верификации

Вывод: FSDP не решает проблему памяти для матриц A



Архитектура AlphaBetaCROWN + auto_LiRPA

auto_LiRPA

Библиотека для автоматического вычисления границ (Linear Relaxation based Perturbation Analysis)

Ключевые компоненты:

- ▶ BoundedModule — обертка над PyTorch моделью
- ▶ Bound операторы для каждого типа слоя
- ▶ Метод bound_backward — обратное распространение границ

Идея реализации

Создать новые классы операторов BoundLinearTP_Col и BoundLinearTP_Row, которые переопределяют bound_backward для распределенных вычислений



Что было сделано: Структура кода

Созданные файлы:

1. `auto_LiRPA/operators/tensor_parallel.py`
 - ▶ `BoundLinearTP_Col` — Column Parallel оператор
 - ▶ `BoundLinearTP_Row` — Row Parallel оператор
2. `examples/simple/tp_model.py`
 - ▶ `ColumnParallelLinear` — PyTorch слой с разделенными весами
 - ▶ `RowParallelLinear` — PyTorch слой с разделенными весами
3. `examples/simple/test_tp_verification.py`
 - ▶ Тестовый скрипт для демонстрации распределенной верификации

Всего: 3 новых модуля, ~400 строк кода



Реализация: BoundLinearTP_Col

Псевдокод Column Parallel слоя

```
def bound_backward(self, last_lA, last_uA, ...):  
    # Вызвать базовый метод для локальных вычислений  
    result = super().bound_backward(...)  
  
    # Извлечь A-матрицы  
    lA_x, uA_x = result[0][0]  
  
    # AllReduce для объединения результатов  
    dist.all_reduce(lA_x, op=ReduceOp.SUM)  
    dist.all_reduce(uA_x, op=ReduceOp.SUM)  
  
    return result
```

Ключевая идея: Использовать `torch.distributed.all_reduce` для суммирования частичных произведений



Реализация: BoundLinearTP_Row

Псевдокод Row Parallel слоя

```
def bound_backward(self, last_lA, last_uA, ...):  
    # Вызвать базовый метод  
    result = super().bound_backward(...)  
  
    # Результат уже корректно разделен!  
    # Коммуникация НЕ нужна  
  
    return result
```

Особенность: Row Parallel слои автоматически производят разделенный результат без дополнительной коммуникации



Статус реализации

Что работает

- ✓ Код компилируется и интегрируется с auto_LiRPA
- ✓ Базовый пример верификации работает на 2 GPU
- ✓ Классы TP операторов реализованы
- ✓ Тестовая модель с TP слоями создана

Что требует доработки

- ▶ Полное тестирование на 2+ GPU
- ▶ Тестирование на больших моделях
- ▶ Интеграция с полным циклом AlphaBetaCROWN
- ▶ Оптимизация коммуникаций (overlapping)



Цель экспериментов

Главный вопрос

Подтвердить, что Tensor Parallelism решает проблему памяти и позволяет верифицировать модели, которые не влезают на один GPU

Гипотеза:

- ▶ Модель размера M вызывает OOM на 1 GPU
- ▶ Та же модель с $TP=N$ успешно верифицируется на N GPU
- ▶ Модель размера $M \times N$ с $TP=N$ использует примерно столько же памяти, как модель M на 1 GPU



Эксперимент 1: Базовый тест памяти

Задача: Найти точку OOM для одного GPU

Метод:

1. Взять простую модель (MLP или WideResNet)
2. Постепенно увеличивать ширину слоев
3. Для каждого размера запускать верификацию и измерять память
4. Найти размер M , при котором происходит OOM

Ожидаемый результат:

- ▶ Определена граница M для одного GPU
- ▶ Установлена baseline для сравнения с TP



Эксперимент 2: TP масштабирование

Задача: Проверить, что TP позволяет верифицировать модель размера $M \times N$ на N GPU

Метод:

1. Взять модель размера M (которая вызывает OOM на 1 GPU)
2. Запустить с TP=2 на 2 GPU
3. Проверить, что верификация проходит успешно
4. Увеличить модель до размера $2M$ с TP=2
5. Измерить потребление памяти на каждом GPU

Ожидаемый результат:

- ▶ Модель M с TP=2 успешно верифицируется
- ▶ Модель $2M$ с TP=2 использует примерно столько же памяти на GPU, как модель M на 1 GPU
- ▶ Доказано, что TP решает проблему масштабируемости



Эксперимент 3: Точность границ

Задача: Убедиться, что распределенное вычисление не влияет на точность границ

Метод:

1. Взять модель, которая влезает на 1 GPU
2. Вычислить границы обычным способом (без TP)
3. Вычислить границы с TP=2
4. Сравнить результаты (должны совпадать с точностью до floating point)

Ожидаемый результат:

- ▶ Границы совпадают с точностью 10^{-6}
- ▶ Распределенное вычисление корректно



Следующие шаги

Краткосрочные задачи

1. Завершить реализацию и тестирование на 2 GPU
2. Провести эксперименты 1-3
3. Интегрировать с полным циклом AlphaBetaCROWN

Долгосрочные задачи (к защите диплома)

1. Оптимизация коммуникаций (overlapping)
2. Тестирование на реальных больших моделях (Transformers, LLM)

